



UNIVERSIDAD AUTÓNOMA DE
CHIHUAHUA



1.5 Actividad 3 documentación

Universidad Autónoma de Chihuahua
Campus 2, Facultad de Ingeniería
Bases de datos avanzadas

Alumno: Alan Fabricio Vega Gutiérrez, Matricula 385658.

Maestro: Jose Saul De Lira Miramontes

26/08/2026

DOCUMENTACIÓN DEL CÓDIGO

Primeramente cabe aclarar que para el funcionamiento correcto de este script se requiere que la tabla “Jobs” sea creada en freesql de antemano.

Importaciones y leer el archivo .env

Se hace uso del módulo “os” de Python, el cual permite leer las variables de entorno usadas para la validación de credenciales de freesql.

También se usa la librería oracledb el cual es el driver oficial de Oracle para Python y permite que el script se comuniquen con la base de datos, luego se importa y usa la función load_dotenv para leer el archivo .env, el cual contiene las credenciales usadas para la conexión con freesql y se encuentra separado del script por seguridad, para usar el script se deberá crear un .env con las credenciales del scheme de freesql.

```
DB_USER = os.getenv("DB_USER")
DB_PASSWORD = os.getenv("DB_PASSWORD")
DB_DSN = os.getenv("DB_DSN")
```

Aquí es donde se leen las credenciales del archivo .env usando las variables de entorno creadas en dicho archivo, aquí es donde se usa os.

Configuración de conexión con freesql

Se define una función llamada conectar, esta incluye un try para evitar el cuelgue del programa si ocurre un error al intentar la conexión con la base de datos, oracledb.connect abre la sesión en la base de datos de Oracle usando las variables extraídas del .env, si todo es correcto se devuelve un objeto de conexión activa que se guarda en la variable “conexión”, si algo sale mal en el proceso de conexión el error es capturado en la variable “e”, que luego se imprime junto a un mensaje de error para notificar al usuario.

```
def conectar():#devuelve una conexion a la base de datos de freesql
    try:
        conexion = oracledb.connect(
            user=DB_USER,
            password=DB_PASSWORD,
            dsn=DB_DSN,
        )
        return conexion
    except oracledb.DatabaseError as e:
        print(f"Error al conectar a la base de datos: {e}")
        return None
```

Create

Se define la función `crear_job`, primeramente, pide al usuario escribir cada uno de los parámetros que se le piden y que se van a agregar a la tabla `Jobs` (`job_id`, `job_title`, `min_salary` y `max_salary`), se usan funciones `strip` para eliminar espacios sobrantes y evitar errores.

`Min_salary` y `max_salary` son opcionales, se valida mediante ifs si estos campos fueron llenados, de lo contrario se envía un `NONE`.

```
conexion = conectar()
if not conexion:
    return
try:#Insertar en jobs
    cursor = conexion.cursor()
    cursor.execute(
        """
        INSERT INTO jobs (job_id, job_title, min_salary, max_salary)
        VALUES (:job_id, :job_title, :min_salary, :max_salary)
        """,
        job_id=job_id,
        job_title=job_title,
        min_salary=min_salary,
        max_salary=max_salary,
    )
    conexion.commit()
    print("Registro insertado correctamente.")
except oracledb.DatabaseError as e:
    print(f"Error al insertar: {e}")
finally:
    cursor.close()
    conexion.close()
```

Esta es la sección donde se suben los datos insertados a la tabla `Jobs`, primeramente, se llama a la función `conectar` mediante la variable “conexión” y usando un cursor, el cual ejecuta los comandos sql, luego se ejecuta el `INSERT` en dos partes, la parte entre “”” usa parámetros nombrados que no son valores directos y luego se pasan los valores reales capturados anteriormente, esto forma una “consulta parametrizada” que se usa por seguridad, finalmente se ejecuta `conexión.commit` para guardar los cambios en la base de datos y dentro de un `finally` se cierran el cursor y la conexión.

Si algo falla en este proceso se lanza una excepción, la cual usa el mismo sistema descrito en la sección anterior.

Read

Read se compone de dos funciones: leer_jobs y buscar_job.

```
def leer_jobs():
    conexion = conectar()
    if not conexion:
        return
    try:
        cursor = conexion.cursor()
        cursor.execute("SELECT job_id, job_title, min_salary, max_salary FROM jobs ORDER BY job_id")
        filas = cursor.fetchall()

        if not filas:
            print("No hay registros en la tabla jobs.")
            return

        print(f"\n{'JOB_ID':<10}{'JOB_TITLE':<35}{'MIN_SALARY':<12}{'MAX_SALARY':<12}")
        print("-" * 69)
        for job_id, job_title, min_salary, max_salary in filas:
            print(f"{'JOB_ID':<10}{'JOB_TITLE':<35}{str(min_salary or ''):<12}{str(max_salary or ''):<12}")
        print()
    except oracledb.DatabaseError as e:
        print(f"Error al leer: {e}")
    finally:
        cursor.close()
        conexion.close()
```

Primeramente, se establecen la conexión y el cursor mediante el proceso descrito anteriormente, luego se ejecuta un select el cual trae las 4 columnas de todos los registros de la tabla para que un fetchall() los recoja, si la lista está vacía se imprime un mensaje para el usuario.

Si se reciben datos, estos se muestran en forma de tabla, la cual está formada por una función for que recorre cada tupla de filas, finalmente se cierran la conexión y el cursor.

Si ocurre un error durante el proceso se imprime un mensaje de error.

```
def buscar_job(job_id):
    conexion = conectar()
    if not conexion:
        return None
    try:
        cursor = conexion.cursor()
        cursor.execute("SELECT job_id, job_title, min_salary, max_salary FROM jobs WHERE job_id = :id", id=job_id)
        return cursor.fetchone()
    except oracledb.DatabaseError as e:
        print(f"Error al buscar: {e}")
        return None
    finally:
        cursor.close()
        conexion.close()
```

Esta función recibe job_id como parámetro, es usada por “actualizar_job” y eliminar_job” para permitir al usuario elegir cual job eliminar, para esto se usa un select filtrado con un where (where job_id = :id), el parámetro :id se sustituye por el valor de la variable job_id, luego la función fetchone() trae un solo registro coincidente.

Se cierran cursor y conexión igual que las demás funciones, adicionalmente, cuando ocurre un error se devuelve un NONE.

Update

```
def actualizar_job():
    job_id = input("Ingrese job_id a actualizar: ").strip()

    registro = buscar_job(job_id)
    if not registro:
        print("No existe un job con ese job_id.")
        return

    print(f"Registro actual: {registro}")
    job_title = input("Nuevo job_title (Enter para no cambiar): ").strip()
    min_salary = input("Nuevo min_salary (Enter para no cambiar): ").strip()
    max_salary = input("Nuevo max_salary (Enter para no cambiar): ").strip()

    campos = []
    valores = {"id": job_id}

    if job_title:
        campos.append("job_title = :job_title")
        valores["job_title"] = job_title
    if min_salary:
        campos.append("min_salary = :min_salary")
        valores["min_salary"] = int(min_salary)
    if max_salary:
        campos.append("max_salary = :max_salary")
        valores["max_salary"] = int(max_salary)

    if not campos:
        print("No se ingreso ningun cambio.")
        return

    consulta = f"UPDATE jobs SET {', '.join(campos)} WHERE job_id = :id"

    conexion = conectar()
    if not conexion:
        return
    try:
        cursor = conexion.cursor()
        cursor.execute(consulta, valores)
        conexion.commit()
        print("Registro actualizado correctamente.")
    except oracledb.DatabaseError as e:
        print(f"Error al actualizar: {e}")
    finally:
        cursor.close()
        conexion.close()
```

Primero se le pide al usuario el registro que quiere actualizar, si no existen id's coincidentes a la proporcionada por el usuario se envía un mensaje, para esto se reutiliza la función `buscar_job`.

Posteriormente se le piden al usuario los valores que busca cambiar, los cuales son opcionales, se usan `strips` para evitar errores por espacios sobrantes, los campos que el usuario llenó se guardan en una lista y "valores" actúa como un diccionario que contiene los valores reales de esos parámetros.

Mediante condicionales se comprueba si el usuario dejó algo en el llenado de datos y se guarda en la lista `campos`, para que luego el valor real se guarde en el diccionario "valores", esto se repite para todos los parámetros, si no se ingresó ningún campo se le informa al usuario.

```
consulta = f"UPDATE jobs SET {'', '.join(campos)} WHERE job_id = :id"
```

Aquí se arma la secuencia sql final tomando la lista de piezas y uniéndolas con comas.

```
conexion = conectar() ..
if not conexion:
    return
try:
    cursor = conexion.cursor()
    cursor.execute(consulta, valores)
    conexion.commit()
    print("Registro actualizado correctamente.")
except oracledb.DatabaseError as e:
    print(f"Error al actualizar: {e}")
finally:
    cursor.close()
    conexion.close()
```

Aquí se conecta con la base de datos y se ejecuta la consulta pasándole el diccionario de valores completo, `oracledb` toma cada parámetro y ejecuta un `commit`, manejando errores y cerrando todo.

Eliminar

```
def eliminar_job():
    job_id = input("Ingrese job_id a eliminar: ").strip()

    registro = buscar_job(job_id)
    if not registro:
        print("No existe un job con ese job_id.")
        return

    confirmar = input(f"Seguro que desea eliminar '{job_id}'? (s/n): ").strip().lower()
    if confirmar != "s":
        print("Operacion cancelada.")
        return

    conexion = conectar()
    if not conexion:
        return
    try:
        cursor = conexion.cursor()
        cursor.execute("DELETE FROM jobs WHERE job_id = :id", id=job_id)
        conexion.commit()
        print("Registro eliminado correctamente.")
    except oracledb.DatabaseError as e:
        print(f"Error al eliminar: {e}")
    finally:
        cursor.close()
        conexion.close()
```

Se le pide al usuario el id del job que quiera borrar, luego reutiliza `buscar_job` para encontrar ese id, si no encuentra ninguno se le informa al usuario, posteriormente se le pide confirmación al usuario utilizando un `if`, se usa `strip` y `lower` para evitar errores por mayúsculas y espacios, si la respuesta no es exactamente "s" se cancela la operación.

Finalmente se usa el mismo patrón de conexión y commit descrito en las funciones anteriores.

Menú principal

```
def menu():
    while True:
        print("1. Create")
        print("2. Read")
        print("3. Update")
        print("4. Delete")
        print("5. Salir")
        opcion = input("Seleccione una opcion: ").strip()

        if opcion == "1":
            crear_job()
        elif opcion == "2":
            leer_jobs()
        elif opcion == "3":
            actualizar_job()
        elif opcion == "4":
            eliminar_job()
        elif opcion == "5":
            print("Saliendo...")
            break
        else:
            print("Opcion invalida, intente de nuevo.")
```

El menú principal consiste en una función while que se repite mientras no se seleccione la opción 5, la cual incluye un break que detiene el ciclo y finaliza el programa, dentro del while se le pide ingresar una de las opciones al usuario, estas se comprueban con ifs, si el usuario selecciona una opción incorrecta (por ejemplo, el numero 6) se le informará que su elección es errónea.

MANUAL DE USUARIO

Esta aplicación sirve para interactuar con la base de datos Jobs de freesql mediante Python y de manera más simple que directamente usando sql.

Preparación

Para poder usar la aplicación usted deberá contar con una cuenta de Oracle en freesql, esto es con el objetivo de obtener un scheme que conecte la aplicación con freesql mediante un archivo .env que contenga su usuario, la contraseña de su scheme y su DSN, capture estos datos dentro del archivo .env y guárdelo dentro de la misma carpeta donde se encuentra la aplicación. Luego cree la tabla Jobs en freesql ejecutando la siguiente consulta:

```
create table jobs(  
    job_id VARCHAR2(10),  
    job_title VARCHAR2(35) CONSTRAINT job_title_nn NOT NULL,  
    min_salary NUMBER(6),  
    max_salary NUMBER(6),  
    CONSTRAINT job_id_pk PRIMARY KEY(job_id)  
);
```

Menú principal

Al ejecutar la aplicación se le mostrará el menú principal con 5 opciones, para elegir una de ellas escriba el número de la opción deseada y presione enter, elegir la opción 5 terminará con la ejecución de la aplicación.

Opción 1: Create

Se le pedirá llenar 4 campos: job_id, job_title, min_salary y max_salary (estos últimos dos solo aceptan números y son opcionales, si desea saltárselos presione enter sin escribir cuando se le pida llenarlos), los campos pueden ser escritos como usted guste, no hay restricciones de

formato, una vez que haya llenado todos los campos estos se guardarán automáticamente en la base de datos, puede agregar tantos Jobs como usted quiera.

Opción 2: Read

Seleccionar esta opción hará que se muestre la tabla guardada en la base de datos.

Opción 3: Update

Al seleccionar esta opción se le pedirá el id del job que desee modificar, una vez que introduzca el id se le pedirá escribir los nuevos job_title, min_salary y max_salary, si usted no desea modificar alguno de estos datos presione enter sin escribir nada cuando se le pida insertar el dato actualizado, una vez que haya terminado la inserción de los datos estos se guardarán en la base de datos automáticamente.

Opción 4: Delete

Al seleccionar esta opción se le pedirá el id del job que desee eliminar, una vez que haya introducido el id y presionado enter se le pedirá confirmación antes de continuar, si desea continuar escriba "s" y presione enter, si desea cancelar el proceso escriba cualquier otra letra y presione enter, si escribió "s" el job se eliminará y si usted escribió cualquier otra letra volverá al menú principal sin que haya cambios.

Opción 5: Salir

Seleccionar la opción 5 detendrá la ejecución de la aplicación, si usted desea volver a usar la aplicación tendrá que volver a correrla.